DOKUMENTASI KODE — FIRMWARE ROBOT

Firmware Robot Feeder Akuatik

Finalnew.ino — Penerima NRF24 & Kendali Zigzag Otomatis (ESP32)

Judul Skripsi	Pengembangan Robot Akuatik untuk Feeder Otomatis di Tambak
Sub-sistem	Robot / Penerima (ESP32 + NRF24L01 + MPU6050 + 2x ESC)
Berkas	Master Code/Robot/New/Finalnew/Finalnew.ino
Bahasa	C++ (Arduino / ESP32)
Tanggal	29 July 2026

Dokumentasi Kode — Firmware Robot `Finalnew.ino`

Dokumen ini menjelaskan baris demi baris logika firmware `Finalnew.ino` yang berjalan di **ESP32** sisi *robot/penerima*. Firmware menerima perintah misi dari transmitter (Raspberry Pi/PC) melalui modul radio **NRF24L01**, lalu menjalankan pola jelajah **zigzag** otomatis untuk menyebar pakan di tambak berbentuk persegi.

---

1. Gambaran Umum

Firmware ini adalah **otak robot**. Tugasnya menerjemahkan satu perintah tingkat-tinggi dari operator — *"tambak ukuran P×L meter, mode AUTO"* — menjadi rangkaian gerak nyata: maju lurus, belok, geser, ulang, sampai seluruh permukaan tambak tersapu.

Alur kerja singkat:

1. Robot menunggu paket misi berformat `P=<panjang>;L=<lebar>;MODE=AUTO` .
2. Dari dimensi tambak, firmware menghitung **jumlah lintasan (lane)** dan **durasi** tiap gerakan.
3. Untuk tiap lane: **maju lurus** dengan penstabil arah (*heading-hold*) berbasis giroskop MPU6050, lalu **U-turn** (dua kali belok 90°) untuk pindah ke lane berikutnya.
4. Selama bergerak, robot terus memantau **detak jantung (heartbeat)** dari transmitter. Bila sinyal hilang > 3 detik, atau menerima paket `STOP` , robot langsung berhenti (*failsafe*).

Sifat penting yang membentuk seluruh desain kode:

- **Komunikasi dua arah half-duplex ("gantian").** Transmitter mengirim perintah/PING; robot menerima, lalu **bergiliran** mengirim balik telemetri MPU (roll/pitch/yaw) di sela-selanya — seperti radio HT. Detailnya di bagian 6 & 11.
- **Gerak berbasis waktu (open-loop), bukan posisi.** Robot tidak punya GPS/encoder. "Maju sekian meter" diwujudkan sebagai "maju selama sekian milidetik", dihitung dari kecepatan asumsi. Karena itu beberapa konstanta **wajib dikalibrasi di air**.
- **Belok differential thrust.** Tidak ada kemudi; belok dilakukan dengan mematikan satu motor dan menjalankan motor lain, sehingga robot **berbelok melingkar lebar**, bukan berputar di tempat.

> Kaitan dengan arahan pembimbing: perintah tingkat-tinggi *"maju x meter, belok y derajat"* saat ini **diterjemahkan oleh robot sendiri** dari dimensi P×L menjadi *timeline* durasi gerak (bagian 10). Jadi pola jalur sudah otomatis; yang belum ada adalah *daftar segmen path eksplisit* dari aplikasi — ini bahan iterasi berikutnya (bagian 13).

---

2. Perangkat Keras & Pustaka

Pustaka yang di-*include* (baris 1–6):

Pustaka	Fungsi
`SPI.h`	Bus SPI untuk komunikasi ke modul NRF24L01
`RF24.h`	Driver modul radio NRF24L01
`Wire.h`	Bus I2C untuk sensor MPU6050
`MPU6050_tockn.h`	Pembacaan sudut (roll/pitch/yaw) dari MPU6050
`ESP32Servo.h`	Menghasilkan sinyal PWM servo 50 Hz untuk ESC brushless
`math.h`	Fungsi `ceil()` dan `fabs()`

Pemetaan pin ESP32:

Komponen	Sinyal	Pin ESP32	Catatan
NRF24L01	CE	GPIO4	`RF24 radio(4, 5)`
NRF24L01	CSN	GPIO5	
NRF24L01	SCK / MOSI / MISO	GPIO18 / 23 / 19	bus VSPI default ESP32
MPU6050	SDA / SCL	GPIO21 / GPIO22	`Wire.begin(21, 22)`
ESC Kiri	Sinyal PWM	GPIO25	`ESC_LEFT_PIN`
ESC Kanan	Sinyal PWM	GPIO26	`ESC_RIGHT_PIN`

> **Jangan** memakai GPIO9/GPIO10 untuk apa pun — keduanya terhubung ke *flash* internal modul WROOM dan akan membuat ESP32 *crash* / *boot-loop* (lihat komentar baris 10–11).

Tentang ESC & motor (baris 22–28): dua *Brushless ESC 30A* tipe RC. Sinyal servo: **1000 µs = stop/arming**, **2000 µs = throttle penuh**. ESC tipe ini **unidirectional** (hanya maju, tidak bisa mundur), itulah sebabnya belokan memakai *differential thrust* dan radius putarnya lebar.

---

3. Konstanta Kalibrasi

Blok `#define` (baris 36–46) adalah tempat tuning utama perilaku robot; nilainya harus disamakan dengan asumsi di sisi transmitter dan **diukur ulang di air**. Sejak protokol diperluas, `ROBOT_SPEED_MPS` dan `LANE_WIDTH_M` berperan sebagai **default** — bisa ditimpa per-misi oleh field `SPD=` / `LW=` dari GUI (lihat bagian 11).

Konstanta	Nilai default	Arti & dampak
`ROBOT_SPEED_MPS`	`0.5`	Kecepatan jelajah robot (m/s) pada `CRUISE_PWM` . Dipakai mengubah jarak -> durasi. **Ukur di air.**
`LANE_WIDTH_M`	`1.0`	Jarak antar lintasan (m). Makin kecil -> makin banyak lane -> sapuan lebih rapat.
`TURN_90_MS`	`1500`	Durasi belok ~90° (ms). Berbasis waktu, **sangat bergantung radius ESC** -> wajib kalibrasi.
`CRUISE_PWM`	`1300`	Throttle jelajah (µs). Menentukan kecepatan nyata dan harus konsisten dengan `ROBOT_SPEED_MPS` .
`KP_HEADING`	`8.0`	Gain proporsional koreksi arah. Terlalu besar -> oleng; terlalu kecil -> lambat lurus.
`MAX_CORRECTION`	`60`	Batas koreksi PWM (µs) agar motor tidak dikoreksi ekstrem.
`HEADING_DEADBAND`	`2.0`	Toleransi error arah (derajat) sebelum koreksi aktif — mencegah getaran di sekitar target.
`SIGNAL_TIMEOUT_MS`	`3000`	Ambang *failsafe*: bila tak ada sinyal selama ini -> robot berhenti.

---

4. Variabel Status Global

Baris 50–63 menyimpan keadaan robot antar-iterasi `loop()` :

Variabel	Tipe	Peran
`panjangTambak` , `lebarTambak`	`int`	Dimensi tambak hasil parsing paket misi
`modeRobot`	`String`	Mode misi (`"AUTO"`)
`missionReceived`	`bool`	Penanda ada misi baru siap dijalankan
`lastSignalMs`	`unsigned long`	Waktu (`millis()`) paket terakhir diterima — dasar *heartbeat*
`stopRequested`	`bool`	`true` saat paket `STOP` diterima di tengah misi
`robotSpeedMps`	`float`	Kecepatan jelajah dari paket `SPD=` (default `ROBOT_SPEED_MPS`)
`laneWidthM`	`float`	Lebar lintasan dari paket `LW=` (default `LANE_WIDTH_M`)
`firstTurnRight`	`bool`	Arah U-turn pertama dari paket `DIR=` (`R -> true`)
`lastTelemetryMs`	`unsigned long`	Waktu terakhir kirim telemetri MPU (throttle, lihat bagian 6)

Selain itu, `const byte telemetryAddress[6] = "00002"` (dekat `address`) adalah **alamat balik** untuk telemetri Robot->Pi, terpisah dari alamat perintah `"00001"` .

---

5. Fungsi Gerak Dasar (Motor Primitives)

Empat fungsi ini adalah lapisan terendah yang langsung menulis sinyal PWM ke ESC.

- `armESC()` (baris 67–74) — mengirim 1000 µs ke kedua ESC lalu `delay(5000)` . **Wajib** dijalankan sekali saat *boot*; tanpa arming, ESC menolak semua perintah throttle.
- `stopBoat()` (baris 76–80) — kedua motor ke 1000 µs (berhenti). Dipakai di semua jalur *failsafe* dan akhir misi.
- `forwardBoat(speedPWM)` (baris 82–87) — kedua motor ke nilai PWM sama (maju lurus tanpa koreksi). *Catatan: pada praktiknya gerak lurus memakai* `driveHeadingMS()` *yang berkoreksi, bukan fungsi ini.*
- `turnLeft()` / `turnRight()` (baris 89–100) — *differential thrust*: satu motor `1000` (stop), satu motor `CRUISE_PWM` (maju). Menghasilkan belok **melingkar lebar** ke arah motor yang berhenti. Dipakai oleh `turn90()` (bagian 8).

---

6. Failsafe & Heartbeat — `signalAlive()`

Baris 139–161. Ini adalah **jaring pengaman** yang dipanggil terus-menerus di dalam setiap gerakan.

```
bool signalAlive() {
if (radio.available()) {
radio.read(buf, ...); // konsumsi paket masuk
lastSignalMs = millis(); // segarkan detak jantung
if (strncmp(buf, "STOP", 4)==0)
stopRequested = true; // STOP di tengah misi -> tandai abort
maybeSendTelemetry(); // GILIRAN KIRIM: balas telemetri MPU
}
return (millis() - lastSignalMs) < SIGNAL_TIMEOUT_MS;
}
```

Konsep heartbeat: transmitter mengirim paket `PING` tiap 0,5 detik selama misi berjalan. Setiap paket apa pun menyegarkan `lastSignalMs` . Bila selisih waktu melebihi `SIGNAL_TIMEOUT_MS` (3 detik),

fungsi mengembalikan `false` -> pemanggil menghentikan robot. Ini melindungi robot agar tidak "kabur" saat radio putus.

6.1 Telemetri MPU balik — `sendTelemetry()` */* `maybeSendTelemetry()`

Baris 110–133. Tepat setelah menerima paket, robot **bergiliran** mengirim data MPU ke Pi — mekanisme *half-duplex* seperti radio HT:

```
void sendTelemetry() {
snprintf(tel, 32, "R=%.1f;P=%.1f;Y=%.1f", roll, pitch, yaw);
radio.stopListening(); // giliran kirim
radio.openWritingPipe(telemetryAddress); // alamat balik "00002"
radio.write(tel, strlen(tel)+1, true); // multicast=true -> NOACK, ~1 ms
radio.startListening(); // langsung dengar perintah lagi
}
```

`maybeSendTelemetry()` membungkusnya dengan *throttle* `TELEMETRY_INTERVAL_MS` (maks ~10 Hz). Kenapa heartbeat **tidak** terganggu:

- Window TX hanya **~1 ms** lalu langsung `startListening()` lagi — robot tidak berdiam di mode kirim.
- Ditulis **NOACK** (`multicast=true` , memerlukan `enableDynamicAck()` di `setup`) -> tanpa retry, tidak *blocking* walau Pi belum sempat mendengar.
- STOP dikirim Pi berulang (5×) selama ~250 ms, sedangkan robot "budek" hanya ~1 ms tiap ~100 ms -> mustahil kelewat semua. Failsafe 3 detik tetap utuh.

Pi menangkap paket ini di alamat `"00002"` sesudah tiap PING. Hook yang sama juga dipasang pada cabang `PING` di `loop()` (bagian 10.1).

---

7. Maju Lurus dengan Heading-Hold — `driveHeadingMS()`

Baris 163–214. **Inti gerak "maju x meter".** Robot maju selama `durationMs` sambil menjaga arah (yaw) tetap di `targetYaw` memakai kontrol **proporsional**.

Cara kerja per iterasi (tiap ~50 ms -> 20 Hz):

1. `mpu.update()` — baca orientasi terbaru.
2. **Cek failsafe:** bila `!signalAlive()` atau `stopRequested` -> `stopBoat()` dan `return false` (abort).
3. Hitung error arah `err = targetYaw - yaw` , lalu **dinormalisasi** ke rentang -180°…+180° (baris 188–189) agar koreksi selalu mengambil arah putar terpendek.
4. Bila `|err| > HEADING_DEADBAND` , hitung koreksi `KP_HEADING * err` , dibatasi `±MAX_CORRECTION` .
5. Terapkan **differential**: motor kiri `CRUISE_PWM + koreksi` , motor kanan `CRUISE_PWM - koreksi` , masing-masing di-*clamp* ke 1000–2000 µs.
6. Cetak status (Yaw/Target/Error/Koreksi) ke Serial untuk *debugging*.

Selesai durasi -> `stopBoat()` dan `return true` . Nilai kembalian ini penting: pemanggil di `loop()` memakainya untuk memutuskan lanjut atau membatalkan misi.

> **Mengapa proporsional (P) saja, bukan PID?** Untuk menjaga arah lurus pada kecepatan rendah, kontrol P sudah cukup dan stabil. `HEADING_DEADBAND` mencegah osilasi kecil di sekitar target.

---

8. Belok 90° — `turn90(toRight)`

Baris 223–247. Menjalankan `turnRight()` **atau** `turnLeft()` sesuai parameter `toRight`, lalu menahannya selama `TURN_90_MS`, sambil tetap mengecek *failsafe* dan `stopRequested` tiap 20 ms. Selesai -> `stopBoat()`. Fungsi ini menggantikan `turnRight90()` lama agar U-turn bisa berbelok ke kiri maupun kanan (dibutuhkan pola zigzag berselang-seling di bagian 10). Di bawahnya, helper `tokenEnd()` (baris 249–253) membantu memisah field paket pada karakter `;`.

> **Keterbatasan penting:** belokan ini **berbasis waktu**, bukan berbasis sudut giroskop. Firmware tidak memverifikasi bahwa robot benar-benar berputar 90°. Karena radius putar ESC unidirectional bervariasi (arus, muatan, arus air), `TURN_90_MS` **harus dikalibrasi** dan tetap rawan galat akumulatif. Perbaikan: gunakan target yaw ±90° dengan *feedback* MPU (bagian 13).

---

9. Inisialisasi — `setup()`

Baris 257–306. **Urutan inisialisasi di sini sengaja dibuat NRF terlebih dahulu** — ini keputusan desain penting (komentar "SOLUSI 5A", baris 261–266). Di sini juga dipanggil `radio.enableDynamicAck()` agar telemetri bisa dikirim NOACK (bagian 6.1):

1. `Serial.begin(115200)`.
2. **Radio NRF di-init paling awal.** Alasannya: `mpu.calcGyroOffsets()` bisa *hang* selamanya bila I2C bermasalah, dan `armESC()` menahan `delay(5000)`. Bila NRF di-init belakangan, `startListening()` baru aktif ~8 detik setelah *boot* (atau tidak sama sekali) -> robot tidak meng-ACK -> transmitter melaporkan **FAILED**. Dengan NRF didahulukan, radio langsung *listening*.
3. Konfigurasi radio: `setPALevel(RF24_PA_LOW)`, `setDataRate(RF24_250KBPS)`, `setChannel(0x76)`, `openReadingPipe(1, address)` dengan alamat `"00001"`, lalu `startListening()`. Bila `radio.begin()` gagal -> loop pesan error tanpa henti (memaksa perbaikan wiring).
4. **MPU6050:** `Wire.begin(21, 22)`, `mpu.begin()`, `mpu.calcGyroOffsets(true)` — kalibrasi offset gyro (robot harus **diam** saat langkah ini).
5. **ESC:** `ESP32PWM::allocateTimer(0..3)` — **wajib** sebelum `attach`, jika tidak PWM bisa gagal diam-diam; `setPeriodHertz(50)`; `attach(pin, 1000, 2000)`; lalu `armESC()`.

> Konfigurasi radio (channel `0x76`, data rate `250KBPS`, PA `LOW`, address `00001`) **harus identik** dengan sisi transmitter, atau kedua modul tidak akan saling terhubung.

---

10. Loop Utama & Algoritma Zigzag — `loop()`

Baris 308–459. Terdiri dari dua bagian: **penerimaan paket** dan **eksekusi misi**.

10.1 Penerimaan & klasifikasi paket (baris 312–351)

Setiap paket yang masuk menyegarkan `lastSignalMs`, lalu diklasifikasikan:

- `PING` -> heartbeat murni. Segarkan waktu, **balas telemetri MPU** (`maybeSendTelemetry()`, bagian 6.1), lalu `return` — **jangan** dianggap misi baru, agar misi berjalan tidak ter-*trigger* ulang.
- `STOP` (saat idle) -> `stopBoat()`, reset `stopRequested = false`, lalu `return`.
- **Selain itu** -> dianggap paket misi. Firmware mem-parse `P=`, `L=`, `MODE=` (wajib) serta `SPD=`, `LW=`, `DIR=` (opsional) dengan `indexOf` / `substring` + `tokenEnd`. Bila format valid -> isi `panjangTambak`, `lebarTambak`, `modeRobot`, `robotSpeedMps`, `laneWidthM`, `firstTurnRight`, set `missionReceived = true`. Bila tidak -> cetak pesan `ERROR: Format tidak valid`. Parameter opsional yang tak ada di-reset ke default konstanta, sehingga paket lama (`P=..;L=..;MODE=AUTO`) tetap jalan.

10.2 Eksekusi misi AUTO (baris 395–458)

Ketika `missionReceived && modeRobot == "AUTO"` (durasi memakai `robotSpeedMps` & `laneWidthM` — nilai dari paket `SPD` / `LW` , atau default konstanta):

```
nLanes = ceil(lebarTambak / laneWidthM) // jumlah lintasan
laneDurationMs = panjangTambak / robotSpeedMps * 1000 // waktu tempuh 1 lane
shiftDurationMs= laneWidthM / robotSpeedMps * 1000 // waktu geser 1 lane
```

Untuk tiap lane:

1. **Re-zero heading:** baca yaw saat ini sebagai `targetYaw` baru. Ini melawan **akumulasi drift giroskop** — tiap lane memakai acuan arah segar, bukan acuan absolut yang makin melenceng.
2. `driveHeadingMS(targetYaw, laneDurationMs)` — maju menyusuri lane. Bila abort -> hentikan seluruh misi.
3. Bila **bukan lane terakhir**, lakukan **U-turn** = `turn90(turnToRight)` -> `driveHeadingMS(shiftYaw, shiftDurationMs)` (geser 1 lebar lane) -> `turn90(turnToRight)` . Dua belok 90° ke **arah sama** = putar balik 180° sambil bergeser satu lane. Setelah tiap U-turn, `turnToRight` **dibalik** (`turnToRight = !turnToRight`), jadi arah belok **berselang-seling** antar-lane. U-turn pertama mengikuti `DIR` dari paket.

> **Perbaikan penting (coverage):** versi sebelumnya selalu belok kanan pada setiap U-turn. Secara geometri itu menggeser robot +x lalu -x bergantian, sehingga robot hanya bolak-balik di dua kolom dan **tidak menyapu seluruh lebar tambak**. Dengan arah belok berselang-seling (kanan-kanan, lalu kiri-kiri, dst), geseran selalu ke sisi yang sama -> sapuan *boustrophedon* menutupi seluruh tambak. Arah belok pada GUI dan firmware kini dihitung dari sumber yang sama sehingga preview jalur = gerak nyata robot.

Pola yang dihasilkan (contoh tambak yang menghasilkan 3 lane):

```
START
|
v
[=========================>] Lane 1 maju + heading-hold
|
| U-turn = belok kanan 90 (x2)
| + geser 1 lebar lane
v
[<=========================] Lane 2 arah berlawanan
|
| U-turn
v
[=========================>] Lane 3
... dst sampai lane ke-nLanes ...
```

Keterangan: `[====>]` = satu lane ditempuh lurus dengan heading-hold; belokan `U-turn` (dua kali belok kanan 90 derajat + geser) menghubungkan lane ganjil dan genap sehingga arah tempuh selang-seling (pola *boustrophedon* / zigzag).

Setiap gerakan menyisipkan cek *failsafe*; bila salah satu mengembalikan `false` , `aborted = true` dan misi dihentikan dengan pesan `MISI DIBATALKAN` . Bila tuntas -> `MISI SELESAI` . Di kedua kasus `stopBoat()` dipanggil.

> `lastSignalMs` dan `stopRequested` **di-reset di awal misi** (baris 409–410) agar sisa `STOP` lama atau jeda sinyal sebelum misi tidak langsung membatalkan misi baru.

---

11. Protokol Paket

Konsisten dengan sisi transmitter (GUI). Paket misi kini **diperluas** dengan parameter opsional (kompatibel-mundur: firmware lama/paket lama tetap jalan karena field tambahan cukup diabaikan / memakai default):

Paket	Arah	Makna
`P=<m>;L=<m>;MODE=AUTO;SPD=<m/s>;LW=<m>;DIR=<R\	L>`	Transmitter -> Robot
`PING`	Transmitter -> Robot	Heartbeat tiap 0,5 s untuk menjaga *failsafe*
`STOP`	Transmitter -> Robot	Batalkan misi berjalan / pastikan motor mati
(auto-ACK)	Robot -> Transmitter	Balasan otomatis lapisan NRF; menandakan robot hidup & terjangkau

Field opsional yang di-parse firmware (bila tidak ada -> pakai konstanta default di bagian 3):

Field	Contoh	Efek di robot
`SPD=`	`SPD=0.5`	Mengganti `robotSpeedMps` -> mempengaruhi durasi tiap lane & geser
`LW=`	`LW=1`	Mengganti `laneWidthM` -> mempengaruhi jumlah lane & durasi geser
`DIR=`	`DIR=R`	Arah U-turn **pertama** (`R` =kanan / `L` =kiri); U-turn berikutnya berselang-seling

Jalur balik (telemetri 2 arah, bagian 6.1). Robot mengirim ke alamat terpisah `"00002"` sesudah menerima paket:

Paket	Arah	Makna
`R=<roll>;P=<pitch>;Y=<yaw>`	Robot -> Transmitter	Telemetri MPU (NOACK), dikirim di alamat `"00002"` secara half-duplex "gantian"

---

12. Panduan Kalibrasi (Ringkas)

1. `ROBOT_SPEED_MPS` **&** `CRUISE_PWM` — jalankan robot lurus pada `CRUISE_PWM` , ukur jarak/waktu tempuh di air, sesuaikan `ROBOT_SPEED_MPS` agar durasi lane akurat.
2. `TURN_90_MS` — perintahkan satu belok, amati sudut nyata, sesuaikan sampai mendekati 90°.
3. `KP_HEADING` **/** `HEADING_DEADBAND` — bila robot oleng saat lurus, kecilkan `KP_HEADING` ; bila lambat mengoreksi, besarkan. Sesuaikan deadband agar tidak bergetar.
4. `LANE_WIDTH_M` — sesuaikan dengan lebar sapuan pakan yang diinginkan.

---

13. Keterbatasan & Iterasi Berikutnya

- **Belok berbasis waktu, bukan sudut.** Ganti `turn90()` timed dengan koreksi menuju target yaw ±90° berbasis MPU agar belokan presisi tanpa kalibrasi rawan.
- **Gerak *open-loop*.** Tanpa GPS/encoder, galat jarak & sudut terakumulasi sepanjang misi panjang. Tambahkan sensor posisi/encoder untuk *closed-loop*.
- **Telemetri 2 arah — sudah ada (MPU).** Robot kini mengirim balik roll/pitch/yaw via radio (bagian 6.1). Iterasi berikutnya: perluas isi telemetri (status komponen, throttle nyata, baterai).

- **Path masih pola sapuan (zigzag).** Sudah membaik: dimensi, kecepatan, lebar lintasan, dan titik awal kini dapat diatur dari aplikasi dan diterjemahkan menjadi daftar segmen "maju x / belok y°" eksplisit, dengan arah belok berselang-seling yang benar. Iterasi berikutnya: menerima *list* waypoint bebas (bukan hanya sapuan persegi) dari aplikasi untuk path arbitrer.